Project 10 (Bonus) — MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions

Type: Single-agent chat assistant (LangGraph graph: checkpointer + long-term store) · **Stack:** Python, LangGraph, `langchain-openai`, `SqliteSaver`, `InMemoryStore`, Pydantic · **Level:** Intermediate

Tagline: A chat assistant with two memory systems — a per-conversation checkpointer and a per-user long-term store — so a brand-new conversation can still know who you are, with a real "forget me."

> Not part of the numbered Doc01-19 arc — a bonus/portfolio project built on 09_langgraph's long-term memory topic. Read Doc09's Core Concepts first.

Charter (what this project is)

Build a chat assistant that remembers you across *separate* conversations, not just within one. It has two memory systems on purpose: a **checkpointer** (keyed by `thread_id`) that holds one conversation's messages, and a **long-term Store** (keyed by `user_id`) that holds durable facts about you, independent of any one conversation. The hard part isn't reading memory back — it's **deciding what's worth remembering** (saying no far more than yes), not duplicating a fact in slightly different words, and offering a real **"forget me"** that deletes everything.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4

The Story — what you're actually building

Most chatbots have the memory of a goldfish between sessions. Close the tab, open a new one, and you're a stranger again — you re-tell it your name, your preferences, everything. A real assistant shouldn't work that way. The second time you talk to it, in a completely new conversation, it should already know you.

That takes **two** kinds of memory, and keeping them straight is the whole design. The first is short-term: the messages of *this* conversation, so it doesn't forget what you said three turns ago. LangGraph's **checkpointer** does that — but it's keyed by `thread_id`, so a brand-new conversation gets a brand-new empty slate. The second is long-term: facts about *you* — your name, your preferences — that should outlive any single conversation. That's the **Store**, keyed by `user_id`, so it doesn't care how many separate threads you've started.

Step 1 proves the limitation first: a normal graph with only a checkpointer, that forgets everything the moment you start a new thread. Step 2 adds the Store and a node that reads it, so a brand-new thread can still know you. Step 3 adds the harder half: deciding, on its own, what's actually worth writing back, and not saving the same fact twice. Step 4 adds forgetting — a real delete, tested by proving a fresh run afterward truly knows nothing.

> **Before you read further — think about it yourself:** for each of these, which memory does it belong in — the checkpointer or the Store? (a) "what did I just ask you?" (b) "my name is Sam" (c) "I think I'll have pasta tonight". If you can't place all three with confidence, settle that before Step 1 — it's the entire architecture.

Where This Fits

A bonus/portfolio project that gives Doc09's long-term memory topic its own hands-on build. What's new here is the two-memory architecture, the *decision* of what's worth keeping (the real design problem), and a privacy-respecting delete — the things a paragraph of advice can't teach.

Project Structure (the professional layout)

Same rule as every project: `main.py` at the repo root, all code in `src/`. No `pyproject`, no `pip install -e`.

The final layout (after Step 4)

```
<repo folder>/ (MemoryKeeper-...-Remembers-You-Across-Sessions/)
├── src/
│   ├── memory/
│   │   ├── __init__.py
│   │   ├── store.py            recall_facts/save_fact/forget_user
│   │   └── writer.py           MemoryDecision, decide_memory()
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              get_model()
│   ├── prompts/
│   │   ├── chat.txt
│   │   └── writer.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
```

```
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py                build_graph(model, store, checkpointer)
│   ├── nodes.py                recall_node, chat_node, write_back_node
│   └── state.py                ChatState
├── tests/
│   ├── integration/
│   │   └── test_live_memory.py real model: recall/write-back/forget
│   └── unit/
│       ├── test_config.py
│       └── test_store.py       recall/save/forget — no model
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

Why each folder exists

- `src/memory/` — the two long-term-memory pieces: the Store helpers (`store.py`) and the "what's worth remembering" decision (`writer.py`).
- `src/models/` — the one file that builds the chat model.
- `src/` root — the graph, its nodes, and the state that flows through them.

How the structure grows, step by step

- **Step 1** builds config/exceptions/logger/prompts-loader, `models/llm`, `state`, `nodes` (`chat_node` only), `graph` (checkpointer only), `main` (hardcoded thread), `test_config`.
- **Step 2** adds `memory/store.py`, a `recall_node`, a real `user_id`, and `test_store`.
- **Step 3** adds `memory/writer.py` (`decide_memory`) and a `write_back_node`.
- **Step 4** adds `forget_user` and a `forget_me` command in `main.py`.

Setup (do this once, before Step 1)

```
cd MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langgraph langgraph-checkpoint-sqlite \
  langchain-openai python-dotenv pydantic
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `*.sqlite`. Every folder under `src/` gets an empty `__init__.py`.

`langgraph-checkpoint-sqlite` is a separate package — it gives you `SqliteSaver`, a checkpointer backed by a real file instead of the in-memory `MemorySaver`. You need the file-backed one: a specific conversation (`thread_id`) should survive a brand-new `python main.py` process.

A Real Example (so this isn't just theory)

- **Run 1, thread A, user `sam`:** "my name is Sam." MemoryKeeper writes a durable `preferred_name` fact.
- **Run 2, a brand-new thread, same `sam`:** "what's my name?" → it still knows you're Sam (the **Store**, not the checkpointer).
- **Thread C:** "I think I'll have pasta tonight." → **not** written to long-term memory (ephemeral, not a durable fact).
- **Thread D:** "actually, call me Sammy." → **updates** the existing name fact, not a second contradictory one.
- **`forget_me`:** deletes everything about `sam`. A brand-new thread afterward has no idea who you are.

How testing works in this project

Two layers, one style. Because this project builds on Doc09 (plain test scripts, not `pytest`), **every test is a plain OK/WRONG script**, run with `python -m`:

- **`tests/unit/`** — no model: `test_config` (settings), `test_store` (recall/save/update-by-key/forget on an `InMemoryStore`).
- **`tests/integration/test_live_memory.py`** — the real proof with the model: a durable fact is saved and recalled in a brand-new thread; an ephemeral line is not saved; `forget_me` leaves a fresh thread knowing nothing.

Each step ends with a **Tests for this step** box: what it checks, why, when it should pass, what a failure means.

Step 1 — A Graph With Only Short-Term (Checkpointer) Memory

What this step does: a normal LangGraph chat graph with only a `SqliteSaver` checkpointer, that remembers within one conversation but forgets the moment you start a new thread.

Why this step matters: you can't appreciate what long-term memory adds until you've felt its absence — this is the baseline every later step measures against.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write config/exceptions/ `utils/logger` / `utils/prompts`, `models/llm` (`get_model`), `src/state.py` (`ChatState`: `messages` with `add_messages`, `user_id`, `recalled`). 2. Write `src/nodes.py` with `chat_node` (system prompt + the conversation → model reply), and `src/graph.py` `build_graph(model, store, checkpointer)` with `START → chat → END`, compiled with the checkpointer (and store, unused yet). 3. Write `main.py`: build the model + `SqliteSaver` (inside a `with` block), a hardcoded `thread_id`, and a terminal loop. Ask your name, then ask it back in the same thread — it remembers. Start a new thread — it forgets.

Your files after Step 1: config, exceptions, `utils/logger`, `utils/prompts`, `models/llm`, `state`, `nodes` (chat only), `graph`, `main`, `tests/unit/test_config.py`.

Tests for this step (`python -m tests.unit.test_config`): a missing/empty setting raises `MissingConfigError`; a present key loads.

Step 2 — Add a Long-Term Store, and a Node That Reads It

What this step does: adds the `InMemoryStore` (keyed by `user_id`) and a `recall_node` that runs before `chat_node`, so a brand-new thread still knows the user.

Why this step matters: this is the fix for Step 1's gap. The checkpointer is keyed by thread on purpose; the Store is keyed by `user_id`, so it doesn't care how many threads that user has started.

What's new vs. Step 1: `memory/store.py`, a `recall_node`, and a real `user_id`. **What stays the same:** `chat_node` and the checkpointer — you're adding a second memory system, not replacing the first.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `memory/store.py`: `namespace_for(user_id)`, `recall_facts(store, user_id)`, `save_fact(store, user_id, key, fact)` (same key = update), `build_store()`. 2. Add `recall_node(state, *, store)` to `nodes.py` — it reads the store and puts the facts into `state["recalled"]`; `chat_node` folds `recalled` into its system prompt. (The store is injected by `compile(store=...)` because the node declares a `store` parameter.) 3. Update `graph.py` to `START → recall → chat → END`, and `main.py` to carry a real `user_id`. Seed a fact by hand, start a brand-new thread, and confirm it's recalled.

Your files after Step 2: + `memory/store.py`, `nodes.py` (recall), `graph.py` (recall edge), `tests/unit/test_store.py`.

Tests for this step (`python -m tests.unit.test_store`): save then recall returns the fact; a second thread (new) still recalls it; saving the same key updates rather than duplicates.

Step 3 — Deciding What's Worth Remembering, and Writing It Back

What this step does: adds a `write_back_node` (after `chat_node`) that decides, itself, whether the user's last message holds a durable fact worth saving — and saves it under a stable key so a rename updates instead of duplicates.

Why this step matters: Step 2 read facts someone seeded by hand. A real system decides for itself what to keep — and the real design problem is saying no far more often than yes.

What's new vs. Step 2: `memory/writer.py` and a `write_back_node`. **What stays the same:** `recall_node` and the checkpointer.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `memory/writer.py`: a `MemoryDecision` Pydantic model (`should_remember`, `key`, `fact`) and `decide_memory(model, message)` = `prompt | model.with_structured_output(MemoryDecision)`. The prompt says: save only durable facts (name, lasting preferences), never ephemeral things (today's dinner); reuse a stable key (`preferred_name`) so a change updates the same fact. 2. Add `write_back_node(state, *, store)` to `nodes.py`: find the last user message, call `decide_memory`, and if `should_remember`, `save_fact` under the returned key. 3. Update `graph.py` to `START → recall → chat → write_back → END`. Test: a name is saved; "pasta tonight" is not; "call me Sammy" updates the existing name.

Your files after Step 3: + `memory/writer.py`, `nodes.py` (write_back), `graph.py`, `prompts/writer.txt`.

Tests for this step: the integration test's write-back + dedup cases (real model).

Step 4 — Forgetting, and a Privacy-Respecting Delete

What this step does: adds `forget_user(store, user_id)` and a `forget_me` command, deleting **every** stored item for that user — proven by a fresh run afterward knowing nothing.

Why this step matters: a system that remembers you must be able to truly forget you. A delete that clears one key but leaves others isn't a real forget.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Add `forget_user(store, user_id)` to `store.py`: `search` the user's namespace for every item, then `delete` each one — don't assume a single key. 2. Add a `forget_me` command to `main.py`'s loop that calls it and confirms. 3. Test: save two facts, forget, confirm the store has none; a brand-new thread recalls nothing.

Your files after Step 4 (final): + `store.py` (`forget_user`), `main.py` (`forget_me`), `test_store.py` (forget case).

Tests for this step: `test_store` 's forget case (all keys gone); the integration test's forget case (fresh thread knows nothing).

Final Deliverable: an assistant with two memory systems — a per-thread checkpointer and a per-user store — that recalls you in a brand-new conversation, decides for itself what's worth keeping, updates a fact instead of duplicating it, and truly forgets you on request.

Run, Test and Configure the Final Project

```
python main.py
python -m tests.unit.test_config
python -m tests.unit.test_store
python -m tests.integration.test_live_memory
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] Step 1's new-thread amnesia was actually observed, not assumed
- [ ] A brand-new thread recalls a fact from the Store, keyed by `user_id`
- [ ] An ephemeral message ("pasta tonight") is **not** written to long-term memory
- [ ] A rename ("call me Sammy") updates the existing fact, not a second one
- [ ] `forget_me` deletes **every** key for that user; a fresh thread then knows nothing
- [ ] You can say which memory any given fact belongs in — checkpointer or Store

Publish It to GitHub

Name the repo `MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions` , add the `README.md` , commit `.env.example` (never `.env`), and push. It's a strong portfolio piece: it shows you understand short-term vs. long-term memory, the decision of what's worth keeping, and a privacy-respecting delete.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full code only if you say **"Show me the solution."**